Single-Cycle RISC-V Processor (RV64I Subset)

Report

Course: Introduction to Processor Architecture (IPA)

Assignment: Sequential (Single-Cycle) Processor Design

Team Members

Jayakanth M V	2025122004
Lavanya Patil	2024112016
Nihaal Appana	2025122007

Contents

1 Introduction **2**
- 1.1 Objectives . . . 2
- 1.2 Supported ISA Subset . . . 2

2 Architecture Overview **2**
- 2.1 High-Level Block Diagram . . . 3
- 2.2 Final RISC V Processor . . . 3
- 2.3 Module Hierarchy . . . 4

3 Module Descriptions **4**
- 3.1 Top-Level Wrapper – `risc_cpu.v` . . . 4
- 3.2 Datapath – `datapath.v` . . . 5
- 3.3 Program Counter – `pc.v` . . . 7
- 3.4 Instruction Memory – `Instruction_Mem.v` . . . 7
- 3.5 Control Unit – `Control_Unit.v` . . . 8
- 3.6 ALU Control – `ALU_Control.v` . . . 9
- 3.7 64-bit ALU – `alu_64_bit.v` . . . 10
- 3.8 Register File – `register_file.v` . . . 11
- 3.9 Immediate Generator – `immediate.v` . . . 12
- 3.10 Multiplexers – `mux2to1.v` . . . 12
- 3.11 PC Adder – `adder.v` . . . 13
- 3.12 Data Memory – `data_memory.v` . . . 14
- 3.13 ALU Sub-Modules – `ALU_Operations.v` . . . 15

4 Branch Logic **15**

5 Test Program **16**
- 5.1 Assembly Listing . . . 16
- 5.2 Instruction Encoding . . . 16

6 Simulation and Verification **17**
- 6.1 Testbench – `seq_tb.v` . . . 17
- 6.2 Expected Register State . . . 19
- 6.3 Expected Memory State . . . 19
- 6.4 Simulation Output . . . 19

7 Design Decisions and Challenges **22**

8 Conclusion **23**

1 Introduction

This report documents the design and implementation of a **single-cycle (sequential) RISC-V processor** targeting the **RV64I** base integer instruction set. The processor is implemented in Verilog HDL and supports a subset of the full ISA sufficient to run arithmetic, logical, memory, and branch operations.

1.1 Objectives

- Implement a functionally correct single-cycle RV64I datapath in Verilog.
- Support the following instructions: `add`, `sub`, `addi`, `and`, `or`, `ld`, `sd`, `beq`.
- Adhere to a **Big-Endian** 64-bit memory architecture.
- Verify correctness via simulation with a provided testbench.

1.2 Supported ISA Subset

Table 1 summarises the eight instructions supported by this processor.

Table 1: Supported RV64I Instructions

Instruction	Type	Opcode (7-bit)	funct3	Operation
`add`	R	`0110011`	`000`	rd = rs1 + rs2
`sub`	R	`0110011`	`000`	rd = rs1 − rs2
`and`	R	`0110011`	`111`	rd = rs1 & rs2
`or`	R	`0110011`	`110`	rd = rs1 | rs2
`addi`	I	`0010011`	`000`	rd = rs1 + imm
`ld`	I	`0000011`	`xxx`	rd = Mem[rs1+imm]
`sd`	S	`0100011`	`xxx`	Mem[rs1+imm] = rs2
`beq`	B	`1100011`	`xxx`	if rs1==rs2: PC+=imm*2

2 Architecture Overview

The single-cycle datapath executes **one instruction per clock cycle**. All combinational logic completes within a single cycle; only the Program Counter (PC), Register File, and Data Memory are clocked elements.

2.1 High-Level Block Diagram

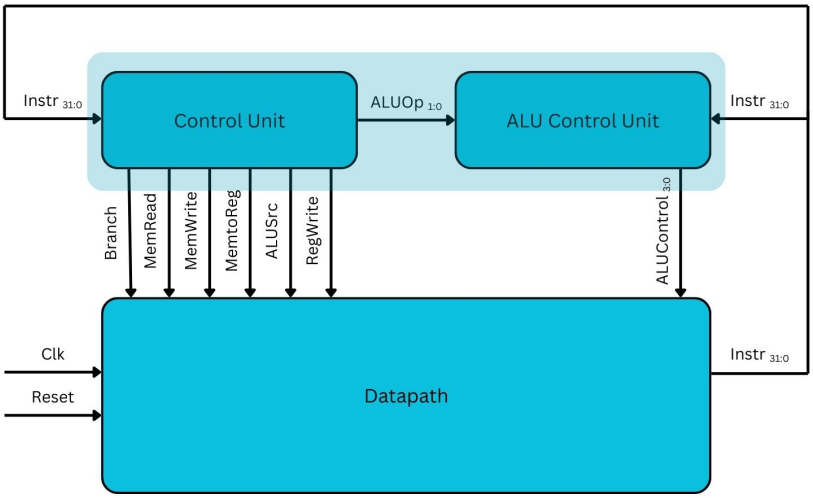

Figure 1: Single-Cycle RV64I Datapath Block Diagram

2.2 Final RISC V Processor

The instruction execution follows the pipeline of stages shown in Figure 2.

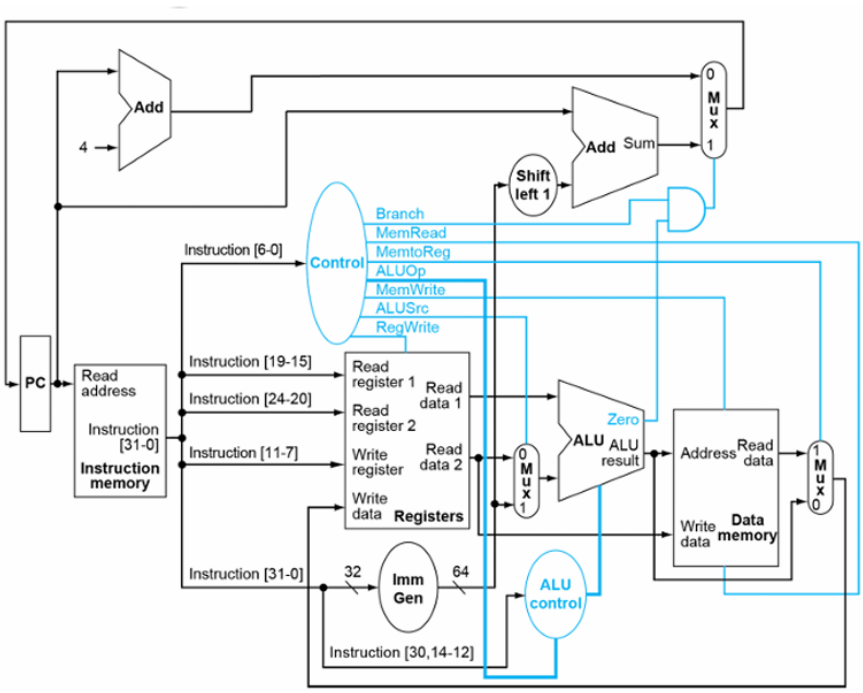

Figure 2: Single-Cycle Instruction Execution Stages

2.3 Module Hierarchy

Table 2 lists the Verilog modules and their roles in the design hierarchy.

Table 2: Module Hierarchy

Module Name	File	Role
RISCV_CORE	risc_cpu.v	Top-level wrapper
datapath	modules/datapath.v	Datapath (instantiates all sub-modules)
pc	modules/pc.v	Program Counter
instruction_mem	modules/Instruction_Mem.v	Instruction Memory
control_unit	modules/Control_Unit.v	Main Control Unit
alu_control	modules/ALU_Control.v	ALU Control
register_file	modules/register_file.v	32×64-bit Register File
immediate_gen	modules/immediate.v	Immediate Generator
mux2 / mux2_64	modules/mux2to1.v	2-to-1 Multiplexer (1-bit / 64-bit)
alu_64_bit	modules/alu_64_bit.v	64-bit ALU
DataMemory	modules/data_memory.v	Data Memory (Big-Endian)
adder	modules/adder.v	64-bit Adder (PC+4 / Branch)
add_1b / add_64b	modules/ALU_Operations.v	Gate-level Adder
xor/or/and_64b	modules/ALU_Operations.v	Bitwise Logic Gates
sll/srl/sra_64b	modules/ALU_Operations.v	Barrel Shifters (L/RL/RA)

3 Module Descriptions

3.1 Top-Level Wrapper – risc_cpu.v

The top-level module RISCV_CORE separates the design into a **control path** (Control Unit + ALU Control) and a **datapath** module. It exposes only two ports: clk and reset.

Table 3: RISCV_CORE Module Ports

Port	Direction	Description
clk	input	System clock
reset	input	Asynchronous active-high reset

Key internal wires include the control signals (Branch, MemRead, MemtoReg, MemWrite, ALUSrc, RegWrite), the 2-bit ALUOp, the 4-bit ALUControl, and the 32-bit Instr bus fed back from the datapath to the control decoders.

```
`include "modules/datapath.v"
`include "modules/Control_Unit.v"
`include "modules/ALU_Control.v"

module RISCV_CORE (
    input clk,
    input reset
```

```
);

  wire Branch, MemRead, MemtoReg, MemWrite, ALUSrc, RegWrite,
     Zero;
  wire [1:0] ALUOp;
  wire [3:0] ALUControl;
  wire [31:0] Instr;

  //CONTROL PATH
    //Control Unit
    control_unit cu1(Instr[6:0], Branch, MemRead, MemtoReg,
                     ALUOp, MemWrite, ALUSrc, RegWrite);

    //ALU Control
    alu_control ac1(ALUOp, Instr[14:12], Instr[31:25], ALUControl
       );

  //DATAPATH
  datapath dp1(clk, reset, Branch, MemRead, MemtoReg,
               ALUControl, MemWrite, ALUSrc, RegWrite, Instr,
                  Zero);

endmodule
```

Listing 1: Top-Level Wrapper (risc_cpu.v)

3.2 Datapath – `datapath.v`

The `datapath` module instantiates all functional units (PC, Instruction Memory, Register File, ALU, Data Memory, Adders, MUXes, Immediate Generator, Shift Left) and wires them together. It receives control signals from the top-level module and outputs `Instr` and `Zero`.

```
module datapath(
    input clk,
    input reset,
    input Branch,
    input MemRead,
    input MemtoReg,
    input [3:0] ALUControl,
    input MemWrite,
    input ALUSrc,
    input RegWrite,
    output [31:0] Instr,
    output Zero
);

  //PC logic
  wire [63:0] pc_in, pc_out;
  pc pc1(clk, reset, pc_in, pc_out);

```

```
  //Instruction Memory
  wire [31:0] ins_temp;
  instruction_mem im1(reset, pc_out, ins_temp);
  assign Instr = ins_temp;

  //Register File
  wire [63:0] write_data, read_data1, read_data2;
  register_file rf1(clk, reset, ins_temp[19:15], ins_temp[24:20],
      ins_temp[11:7], write_data, RegWrite, read_data1,
         read_data2);

  //Immediate Generation
  wire [63:0] ImmExt;
  immediate_gen ig1(ins_temp, ImmExt);

  //MUX: ALUSrc
  wire [63:0] ALU_ip;
  mux2_64 md1(ALUSrc, read_data2, ImmExt, ALU_ip);

  //ALU Block
  wire [63:0] ALU_Res;
  wire cout, carry_flag, overflow_flag, zero_flag;
  alu_64_bit alu1(read_data1, ALU_ip, ALUControl,
      ALU_Res, cout, carry_flag, overflow_flag, zero_flag);
  assign Zero = zero_flag;

  //Data Memory
  wire [63:0] Data_memory_op;
  DataMemory dm1(clk, reset, ALU_Res, read_data2,
      MemRead, MemWrite, Data_memory_op);

  //MUX: MemtoReg
  mux2_64 md2(MemtoReg, ALU_Res, Data_memory_op, write_data);

  //PC + 4
  wire [63:0] pc_update;
  adder a1(pc_out, 64'd4, pc_update);

  //Branch: Shift Left by 1 + PC
  wire [63:0] pc_add, shift_imm;
  sll_64b sl1(ImmExt, 6'b000001, shift_imm);
  adder a2(pc_out, shift_imm, pc_add);

  //MUX: PCSrc
  wire pc_con;
  assign pc_con = Branch & Zero;
  mux2_64 md3(pc_con, pc_update, pc_add, pc_in);

endmodule
```

Listing 2: Datapath Module (datapath.v)

3.3 Program Counter – `pc.v`

The PC is a 64-bit register updated on the rising edge of the clock. An asynchronous reset drives it to zero.

Table 4: `pc` Module Ports

Port	Width	Dir	Description
`clk`	1	in	Clock
`reset`	1	in	Async reset
`pc_in`	64	in	Next PC value
`pc_out`	64	out	Current PC value

```
module pc (
    input          clk,
    input          reset,
    input  [63:0] pc_in,
    output reg [63:0] pc_out
);
    always @(posedge clk or posedge reset) begin
        if (reset)
            pc_out <= 64'd0;
        else
            pc_out <= pc_in;
    end
endmodule
```

Listing 3: Program Counter (pc.v)

3.4 Instruction Memory – `Instruction_Mem.v`

A byte-addressable ROM of 4096 bytes (`IMEM_SIZE = 4096`). Instructions are fetched in Big-Endian order (MSB first) using a combinational `always @(*)` block. The memory is initialised from `instructions.txt` using `$readmemh`.

Table 5: `instruction_mem` Module Ports

Port	Width	Dir	Description
`reset`	1	in	Combinational reset (output → 0)
`addr`	64	in	Byte address (= PC)
`instr`	32	out	32-bit instruction word

```
`define IMEM_SIZE 4096

module instruction_mem(
    input reset,
    input [63:0] addr,
    output reg [31:0] instr
```

```
);

  reg [7:0] RAM[0:`IMEM_SIZE-1];

  initial begin
    $readmemh("instructions.txt",RAM);
  end

  always @(*)begin
    if(reset==1'b1)
      instr = 32'b0;
    else
      instr = {RAM[addr],RAM[addr+1],RAM[addr+2],RAM[addr+3]};
  end

endmodule
```

Listing 4: Instruction Memory (Instruction_Mem.v)

Design note: The instruction fetch uses {`RAM[addr]`, `RAM[addr+1]`, `RAM[addr+2]`, `RAM[addr+3]`}, preserving the Big-Endian byte order. The read is now fully combinational (no clock input).

3.5 Control Unit – `Control_Unit.v`

A purely combinational module that decodes the 7-bit opcode and issues the seven control signals required by the datapath.

Table 6: Control Unit Truth Table

Instruction	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
R-type (`add/sub/and/or`)	0	0	1	0	0	0	10
I-type load (`ld`)	1	1	1	1	0	0	00
I-type ALU (`addi`)	1	0	1	0	0	0	00
S-type (`sd`)	1	X	0	0	1	0	00
B-type (`beq`)	0	X	0	0	0	1	01

X = don't care

```
module control_unit(
  input [6:0] opcode,
  output reg Branch,
  output reg MemRead,
  output reg MemtoReg,
  output reg [1:0] ALUOp,
  output reg MemWrite,
  output reg ALUSrc,
  output reg RegWrite
);
  always @(*) begin
    case(opcode)
      7'b0110011 : begin  // R-type
```

```
        ALUSrc=1'b0; MemtoReg=1'b0; RegWrite=1'b1;
        MemRead=1'b0; MemWrite=1'b0; Branch=1'b0; ALUOp=2'b10;
      end
      7'b0000011 : begin  // I-type load (ld)
        ALUSrc=1'b1; MemtoReg=1'b1; RegWrite=1'b1;
        MemRead=1'b1; MemWrite=1'b0; Branch=1'b0; ALUOp=2'b00;
      end
      7'b0100011 : begin  // S-type (sd)
        ALUSrc=1'b1; MemtoReg=1'bx; RegWrite=1'b0;
        MemRead=1'b0; MemWrite=1'b1; Branch=1'b0; ALUOp=2'b00;
      end
      7'b1100011 : begin  // B-type (beq)
        ALUSrc=1'b0; MemtoReg=1'bx; RegWrite=1'b0;
        MemRead=1'b0; MemWrite=1'b0; Branch=1'b1; ALUOp=2'b01;
      end
      7'b0010011 : begin  // I-type ALU (addi)
        ALUSrc=1'b1; MemtoReg=1'b0; RegWrite=1'b1;
        MemRead=1'b0; MemWrite=1'b0; Branch=1'b0; ALUOp=2'b00;
      end
      default: begin
        ALUSrc=1'b0; MemtoReg=1'b0; RegWrite=1'b0;
        MemRead=1'b0; MemWrite=1'b0; Branch=1'b0; ALUOp=2'b00;
      end
    endcase
  end
endmodule
```

Listing 5: Control Unit (Control_Unit.v)

3.6 ALU Control – `ALU_Control.v`

A two-level decoder that translates the 2-bit `ALUOp` signal (from the Control Unit) together with `funct7[30]` and `funct3[2:0]` (from the instruction) into a 4-bit `ALUControl` code.

Table 7: ALU Control Encoding

ALUOp	funct7[30]	funct3	ALUControl (operation)
00	–	—	0000 (ADD) – load/store address
01	–	—	1000 (SUB) – branch comparison
10	0	000	0000 (ADD)
10	1	000	1000 (SUB)
10	–	111	0111 (AND)
10	–	110	0110 (OR)

```
module alu_control(
  input [1:0] ALUOp,
  input [2:0] funct3,
  input [6:0] funct7,
```

```
  output reg [3:0] ALUControl
);
  always @(*)begin
    case(ALUOp)
      2'b00 : ALUControl = 4'b0000;
      2'b01 : ALUControl = 4'b1000;
      2'b10 : begin
        case(funct3)
          3'b111 : ALUControl = 4'b0111;
          3'b110 : ALUControl = 4'b0110;
          3'b000 : begin
            case(funct7)
              7'b0000000 : ALUControl = 4'b0000;
              7'b0100000 : ALUControl = 4'b1000;
              default    : ALUControl = 4'b0000;
            endcase
          end
          default : ALUControl = 4'b0000;
        endcase
      end
      default : ALUControl = 4'b0000;
    endcase
  end
endmodule
```

Listing 6: ALU Control (ALU_Control.v)

3.7 64-bit ALU – `alu_64_bit.v`

The ALU supports ten operations selected by a 4-bit opcode. It is built from gate-level sub-modules defined in `ALU_Operations.v` (`add_64b`, `sll_64b`, `srl_64b`, `sra_64b`, `xor_64b`, `or_64b`, `and_64b`).

Table 8: ALU Opcode Table

ALU opcode (4-bit)	Operation
0000	ADD
0001	SLL (shift left logical)
0010	SLT (signed less-than)
0011	SLTU (unsigned less-than)
0100	XOR
0101	SRL (shift right logical)
0110	OR
0111	AND
1000	SUB
1101	SRA (shift right arithmetic)

Zero flag: The zero flag is computed using `~|result` (reduction NOR). If the result is all-zero the flag is asserted, used by the `beq` branch logic.

3.8 Register File – `register_file.v`

A 32 × 64-bit register file. Register `x0` is hardwired to zero. Reads are *asynchronous*; writes are *synchronous* (rising-edge clocked).

Table 9: `register_file` Module Ports

Port	Width	Dir	Description
`clk`	1	in	Clock
`reset`	1	in	Async reset (all regs → 0)
`read_reg1`	5	in	Source register 1 address
`read_reg2`	5	in	Source register 2 address
`write_reg`	5	in	Destination register address
`write_data`	64	in	Data to write
`reg_write_en`	1	in	Write enable
`read_data1`	64	out	Data from rs1
`read_data2`	64	out	Data from rs2

```
module register_file (
    input          clk,
    input          reset,
    input  [4:0]   read_reg1,
    input  [4:0]   read_reg2,
    input  [4:0]   write_reg,
    input  [63:0]  write_data,
    input          reg_write_en,
    output [63:0]  read_data1,
    output [63:0]  read_data2
);

    reg [63:0] registers [0:31];
    integer i;

    // Asynchronous read -- x0 always returns 0
    assign read_data1 = (read_reg1 == 5'd0) ? 64'd0 : registers[
       read_reg1];
    assign read_data2 = (read_reg2 == 5'd0) ? 64'd0 : registers[
       read_reg2];

    // Synchronous write with async reset
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            for (i = 0; i < 32; i = i + 1)
                registers[i] <= 64'd0;
        end else begin
            if (reg_write_en && write_reg != 5'd0)
                registers[write_reg] <= write_data;
        end
    end

```

```
endmodule
```

Listing 7: Register File (register_file.v)

3.9 Immediate Generator – `immediate.v`

Performs sign-extension for I-type, S-type, and B-type instructions.

Table 10: Immediate Encoding by Instruction Type

ImmSrc	Type	Bit Extraction
2'b00	I-type	{{52{Instr[31]}}, Instr[31:20]}
2'b01	S-type	{{52{Instr[31]}}, Instr[31:25], Instr[11:7]}
2'b10	B-type	{{52{Instr[31]}}, Instr[7], Instr[30:25], Instr[11:8]}

```
module immediate_gen (
    input [31:0] Instr,
    output reg [63:0] ImmExt
);
    wire [6:0] opcode;
    reg [1:0] ImmSrc;
    assign opcode = Instr[6:0];

    always @(*) begin
        ImmSrc = 2'b00;
        ImmExt = 64'b0;

        case (opcode)
            7'b0000011, 7'b0010011 : ImmSrc = 2'b00; // I type
            7'b0100011             : ImmSrc = 2'b01; // S type
            7'b1100011             : ImmSrc = 2'b10; // B type
        endcase

        case (ImmSrc)
            2'b00 : ImmExt = {{52{Instr[31]}},
                              Instr[31:20]};
            2'b01 : ImmExt = {{52{Instr[31]}},
                              Instr[31:25], Instr[11:7]};
            2'b10 : ImmExt = {{52{Instr[31]}},
                              Instr[7], Instr[30:25], Instr
                                 [11:8]};
        endcase
    end
endmodule
```

Listing 8: Immediate Generator (immediate.v)

3.10 Multiplexers – `mux2to1.v`

The file defines two modules:

- `mux2`: Gate-level 1-bit 2-to-1 MUX (used inside barrel shifters).
- `mux2_64`: Behavioural 64-bit 2-to-1 MUX (`assign out = sel ? in1 : in0`).

Three instances of `mux2_64` are used in the datapath:

- **md1 (ALUSrc)**: Selects between `read_data2` (register) and `ImmExt` (immediate) as ALU operand B.
- **md2 (MemtoReg)**: Selects between `ALU_Res` and `Data_memory_op` for write-back.
- **md3 (PCSrc)**: Selects between `pc_update` and `pc_add` as the next PC.

```
module mux2 (
    input s,
    input i0,
    input i1,
    output y
);
    wire t0, t1, t2;
    not(t0, s);
    and(t1, i0, t0);
    and(t2, i1, s);
    or(y, t1, t2);
endmodule

module mux2_64 (
    input sel,
    input [63:0] in0,
    input [63:0] in1,
    output [63:0] out
);
    assign out = sel ? in1 : in0;
endmodule
```

Listing 9: Multiplexer Modules (mux2to1.v)

3.11 PC Adder – `adder.v`

A simple 64-bit combinational adder (`assign result = in1 + in2`). Two instances are used in the datapath:

1. `a1`: Computes PC + 4.
2. `a2`: Computes PC + imm × 2 (branch target).

```
module adder (
    input  [63:0] in1,
    input  [63:0] in2,
    output [63:0] result
);
    assign result = in1 + in2;
endmodule
```

Listing 10: 64-bit Adder (adder.v)

3.12 Data Memory – `data_memory.v`

A byte-addressable 1024-byte SRAM.

- **Word width**: 8-bit cells; 64-bit words accessed across 8 consecutive bytes.
- **Byte order**: **Big-Endian** – MSB stored at the lowest address.
- **Write**: Synchronous on rising edge when `MemWrite = 1`.
- **Read**: **Asynchronous** (combinational) when `MemRead = 1`.
- **Init**: Loaded from `data_mem.txt` via `$readmemh`.

Table 11: `DataMemory` Module Ports

Port	Width	Dir	Description
`clk`	1	in	Clock
`reset`	1	in	Async reset (clears all bytes)
`address`	64	in	Byte address (64-bit from ALU result)
`write_data`	64	in	Data to write
`MemRead`	1	in	Asynchronous read enable
`MemWrite`	1	in	Synchronous write enable
`read_data`	64	out	Data read from memory

```verilog
`define MEM_SIZE 1024

module DataMemory (
    input          clk,
    input          reset,
    input   [63:0] address,
    input   [63:0] write_data,
    input          MemRead,
    input          MemWrite,
    output reg [63:0] read_data
);
    reg [7:0] mem [0:`MEM_SIZE-1];
    integer i;

    initial begin
        $readmemh("data_mem.txt", mem);
    end

    // Asynchronous read (Big-Endian)
    always @(*) begin
        if (MemRead)
            read_data = {mem[address], mem[address+1],
                         mem[address+2], mem[address+3],
                         mem[address+4], mem[address+5],
                         mem[address+6], mem[address+7]};
        else
            read_data = 64'b0;
```

```
    end

    // Synchronous write with async reset
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            for (i = 0; i < `MEM_SIZE; i = i + 1)
                mem[i] <= 8'b0;
        end else if (MemWrite) begin
            mem[address]   <= write_data[63:56];
            mem[address+1] <= write_data[55:48];
            mem[address+2] <= write_data[47:40];
            mem[address+3] <= write_data[39:32];
            mem[address+4] <= write_data[31:24];
            mem[address+5] <= write_data[23:16];
            mem[address+6] <= write_data[15:8];
            mem[address+7] <= write_data[7:0];
        end
    end
endmodule
```

Listing 11: Data Memory (data_memory.v)

3.13 ALU Sub-Modules – `ALU_Operations.v`

All gate-level arithmetic and logic building blocks are consolidated in a single file. The modules are:

- `add_1b`: 1-bit full adder (gate-level XOR/AND/OR).
- `add_64b`: 64-bit ripple-carry adder built from `add_1b` via `generate`. Outputs `sum`, `cout`, and `c63` (carry into MSB, for overflow detection).
- `xor_64b`, `and_64b`, `or_64b`: Bitwise 64-bit logic using gate primitives in a `generate` loop.
- `sll_64b`: Logical left shift via 6-stage MUX tree (using `mux2` from `mux2to1.v`).
- `srl_64b`: Logical right shift (zero-fills upper bits).
- `sra_64b`: Arithmetic right shift (sign-extends MSB).

Subtraction in the ALU is implemented by inverting `b` (`~b`) and setting `cin = 1` on `add_64b`. SLT uses the sign bit of the result corrected for overflow; SLTU checks the absence of a borrow (`~cout`).

4 Branch Logic

The `beq` instruction computes the branch target as:

$$\text{branch_address} = \text{PC} + \text{imm} \times 2$$

This is realised in hardware as:

1. The Immediate Generator outputs a sign-extended B-type immediate (`{{52{Instr[31]}}, Instr[7], Instr[30:25], Instr[11:8]}`).
2. A **left barrel shifter** (`sll`, instance of `sll_64b`) shifts `ImmExt` left by 1 (multiply by 2), yielding `shift_imm`.
3. The branch adder (`a2`) computes PC + shift_imm, producing `pc_add`.
4. The branch is taken when: `pc_con = Branch & Zero`.
5. MUX `md3` (PCSrc) selects `pc_add` when `pc_con = 1`, otherwise `pc_update` (PC+4).

5 Test Program

5.1 Assembly Listing

Table 12 describes the test program loaded via `instructions.txt`. The program computes the first five Fibonacci numbers ($F_0 = 0, F_1 = 1, F_2 = 1, F_3 = 2, F_4 = 3$) and stores them into data memory at 8-byte intervals.

Table 12: Test Program – Assembly Instructions

PC	Instruction	Operation	Notes
0	`addi x10, x0, 0`	x10 = 0	Memory pointer
4	`addi x5, x0, 0`	x5 = 0	F_0
8	`addi x6, x0, 1`	x6 = 1	F_1
12	`sd x5, 0(x10)`	Mem[0] = 0	Store F_0
16	`sd x6, 8(x10)`	Mem[8] = 1	Store F_1
20	`addi x8, x0, 3`	x8 = 3	Loop counter
— Loop body (PC 24–52) —			
24	`add x7, x5, x6`	x7 = x5 + x6	Next Fibonacci
28	`sd x7, 16(x10)`	Mem[x10+16] = x7	Store F_n
32	`add x5, x6, x0`	x5 = x6	Shift window
36	`add x6, x7, x0`	x6 = x7	Shift window
40	`addi x10, x10, 8`	x10 += 8	Advance pointer
44	`addi x8, x8, -1`	x8 -= 1	Decrement counter
48	`beq x8, x0, 8`	if x8==0: PC = 56	Exit loop
52	`beq x0, x0, -28`	PC = 24 (always)	Loop back
56	`(halt)`	0x00000000	End of program

5.2 Instruction Encoding

The instructions are stored as a sequence of raw hex bytes in `instructions.txt` (Big-Endian, one byte per line). The first instruction `addi x10, x0, 0` encodes as:

`00 00 05 13` $\Rightarrow$ `0x00000513`

Table 13 shows the bit-field breakdown for each R-type and I-type encoding pattern.

Table 13: RISC-V Instruction Format Bit Fields

Type	[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]
R	funct7	rs2	rs1	funct3	rd	opcode
I	imm[11:0]		rs1	funct3	rd	opcode
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B	imm[12,10:5]	rs2	rs1	funct3	imm[4:1,11]	opcode

6 Simulation and Verification

6.1 Testbench – `seq_tb.v`

The testbench performs the following:

1. Generates a 10 ns clock (`always #5 clk = ~clk`).
2. Holds `reset = 1` for 6 ns, then deasserts it.
3. Monitors the `Instr` bus on the negative clock edge; terminates simulation when `32'h00000000` or `32'hxxxxxxxx` is detected.
4. Safety timeout after 10 000 positive clock edges.
5. On termination, calls `dump_registers()` and `save_memory_to_file()` tasks (defined in the testbench) to write `register_file.txt` and `data_mem.txt`.
6. Generates a VCD waveform file (`processor.vcd`).

```
`include "risc_cpu.v"
`timescale 1ns / 1ps

module seq_tb;
    reg clk;
    reg reset;
    integer cycle_count;

    // Instantiate processor
    RISCV_CORE uut (
        .clk   (clk),
        .reset (reset)
    );

    // Clock generation: 10ns period
    initial clk = 0;
    always #5 clk = ~clk;

    // Cycle counter
    initial cycle_count = 1;
    always @(posedge clk) begin
        if (reset)  cycle_count <= 0;
        else        cycle_count <= cycle_count + 1;
    end
```

```

    // Main test sequence
    initial begin
        $dumpfile("processor.vcd");
        $dumpvars(0, seq_tb);

        reset = 1;
        #3;
        reset = 0;

        // Safety timeout
        repeat (10000) @(posedge clk);
        $display("WARNING: Simulation timeout.");
        $finish;
    end

    always @(negedge clk) begin
        if (!reset) begin
            if (uut.Instr === 32'h00000000 ||
                uut.Instr === 32'hxxxxxxxx) begin
                $display("Halt Detected at PC: %h",
                         uut.dp1.pc_out);
                $display("Simulation complete after "
                         "%0d clock cycles", cycle_count);
                dump_registers(cycle_count);
                save_memory_to_file();
                $finish;
            end
        end
    end

    task save_memory_to_file;
        integer file, i;
        begin
            file = $fopen("data_mem.txt", "w");
            for (i = 0; i < 1024; i = i + 1)
                $fwrite(file, "%02x\n",
                        uut.dp1.dm1.mem[i]);
            $fclose(file);
        end
    endtask

    task dump_registers;
        input integer final_count;
        integer fd, j;
        begin
            fd = $fopen("register_file.txt", "w");
            for (j = 0; j < 32; j = j + 1) begin
                if (j == 0)
                    $fwrite(fd, "%016h\n", 64'd0);
                else
```

```
                        $fwrite(fd, "%016h\n",
                                uut.dp1.rf1.registers[j]);
                end
                $fwrite(fd, "%0d\n", final_count);
                $fclose(fd);
            end
        endtask
endmodule
```

Listing 12: Testbench (seq_tb.v)

6.2 Expected Register State

Table 14 shows the expected register values at end of simulation (after 3 loop iterations, 30 clock cycles).

Table 14: Expected Final Register File State

Register	Expected Value (hex)	Notes
x0	0000000000000000	Hardwired zero
x5	0000000000000002	$F_3 = 2$ (prev Fibonacci)
x6	0000000000000003	$F_4 = 3$ (curr Fibonacci)
x7	0000000000000003	$F_4 = 3$ (last computed)
x8	0000000000000000	Loop counter (decremented to 0)
x10	0000000000000018	Memory pointer ($24 = 3 \times 8$)
All other registers remain 0.		

6.3 Expected Memory State

The first five Fibonacci numbers are stored at 8-byte intervals in data memory (Big-Endian, 64-bit words):

Table 15: Expected Data Memory (Fibonacci Sequence)

Address	Value	Fibonacci
0	0	F_0
8	1	F_1
16	1	F_2
24	2	F_3
32	3	F_4

6.4 Simulation Output

Figures 3 and 4 show the simulation output captured after running the Fibonacci test program.

Figure 3: Final Data Memory Output – Fibonacci Sequence

Figure 4: Final Register File State

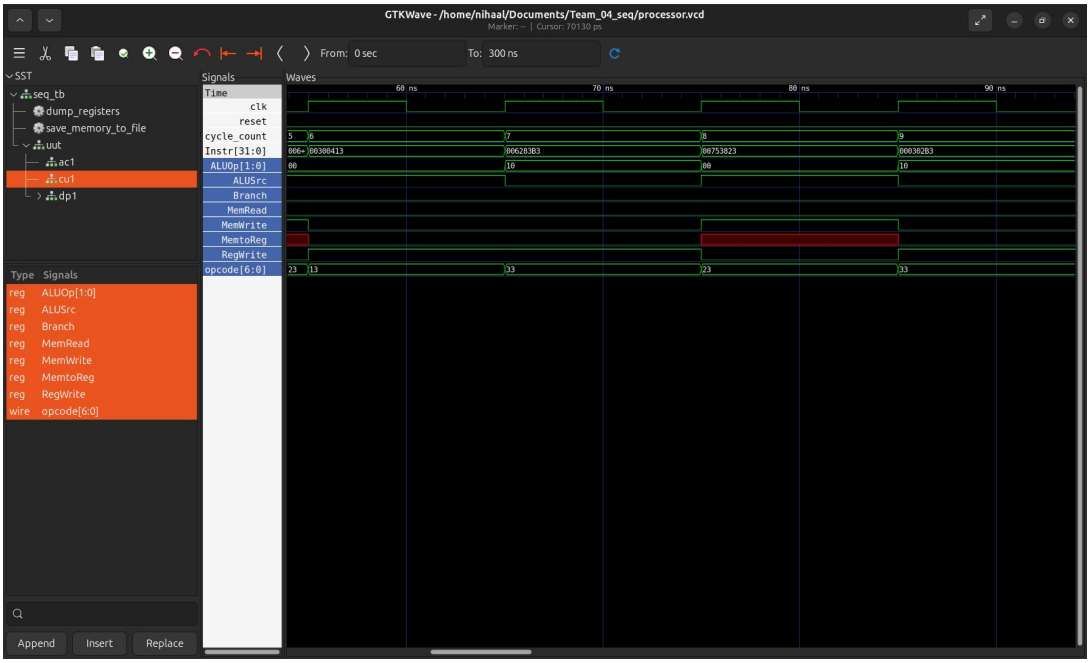

Figure 5: Control Unit – Simulation Output

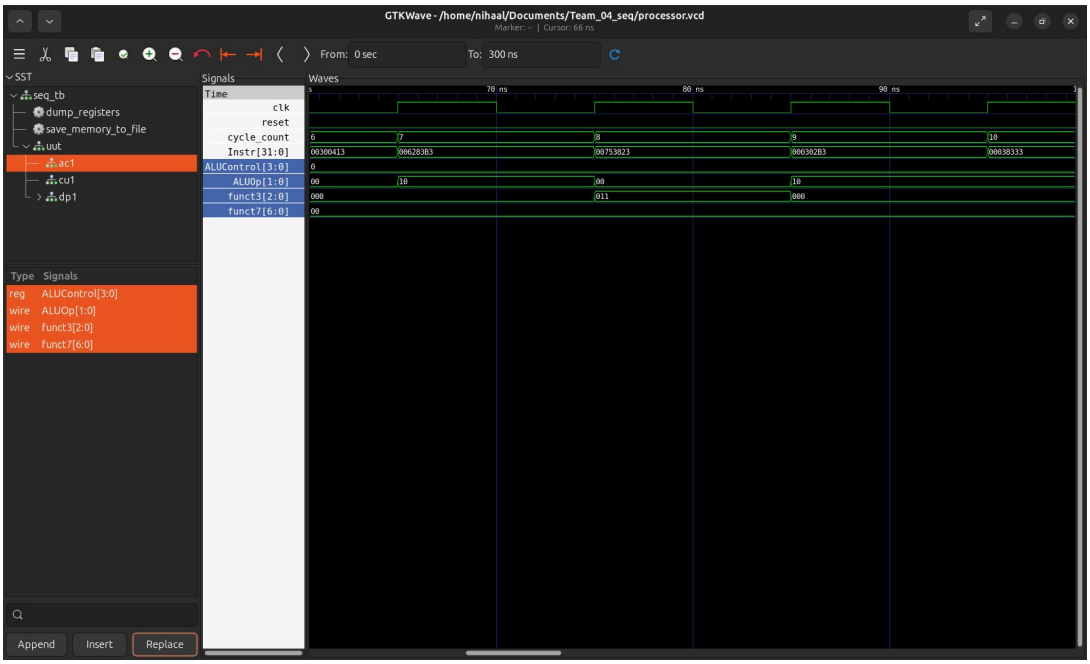

Figure 6: ALU Control – Simulation Output

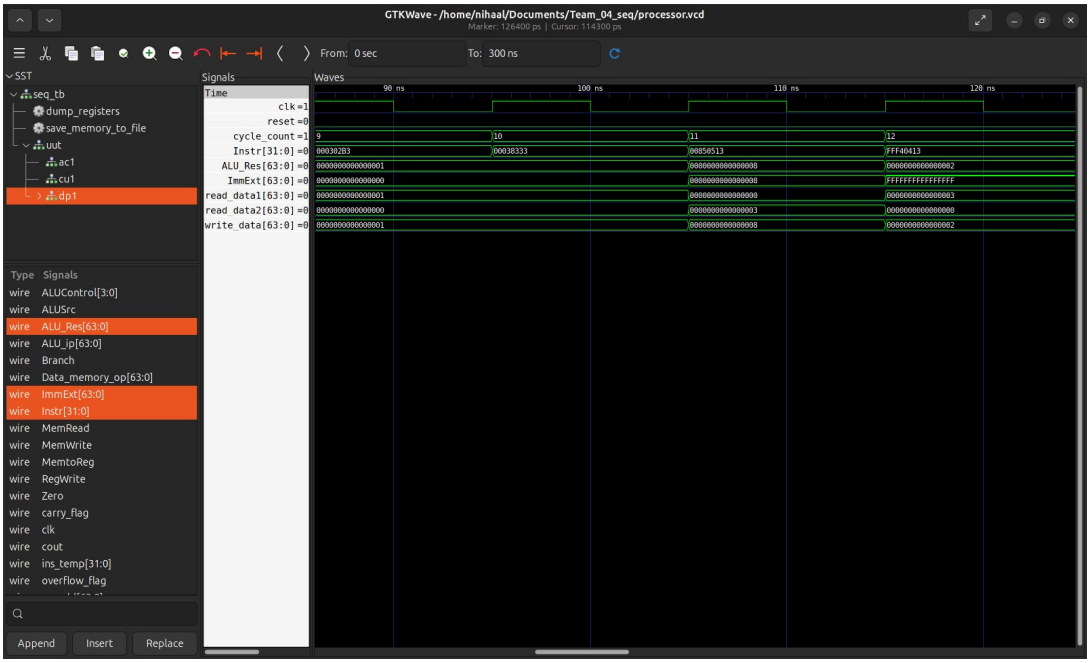

Figure 7: Datapath – Simulation Output

7 Design Decisions and Challenges

1. **Big-Endian Memory**: Both instruction and data memories store the most-significant byte at the lowest address. The fetch logic concatenates `RAM[addr]`, `RAM[addr+1]`, `RAM[addr+2]`, `RAM[addr+3]` to form a 32-bit instruction word.

2. **Branch Offset Multiplication**: The RISC-V B-type immediate already omits the lowest bit (all branches are to even addresses). The `immediate_gen` module right-shifts by 1 (`>> 1`) to store a 12-bit value, and the branch hardware left-shifts by 1 to recover the byte offset.

3. **Asynchronous vs. Synchronous Elements**: Reads from the register file and data memory are asynchronous (combinational) to keep all reads within a single clock cycle. Only PC updates and memory writes are synchronous (registered on rising clock edge).

4. **Gate-Level ALU**: The ALU is built entirely from gate-level primitives (`and`, `or`, `xor`, `buf`, `not`) and structural module instantiation, satisfying the gate-level design constraint in the assignment.

5. **Zero Flag Logic**: The zero flag for the ALU result is computed using a tree of OR gates (`or_64bit_itself`), inverted by a single NOT gate, to avoid any behavioural `always` assignment in the flag path.

6. **ALU Control Mismatch**: In an earlier assignment, we had been asked to implement an ALU with arbitrary ALUControl ops that were given by the evaluators. While implementing the current project, we ended up forgetting about that fact and blindly included the then designed ALU into our RISC wrapper. It took about a day of debugging to realize that the reason nothing was working as it should have

was because of a silly ALUControl mismatch - and not the 50 others things in the wrapper that we initially suspected.

7. **Group Naming Inconsistencies**: While not in the current final wrapper, an earlier iteration was a little messy for the three of us ended up using different names for gates (for example multiplexers). Although it was a relatively easy problem to fix, it was an interesting learning experience. We realized that we should have set some ground rules and naming conventions before splitting the tasks and diving into the implementation.

8 Conclusion

This report has documented the complete implementation of a single-cycle RV64I processor in Verilog HDL. The design correctly implements the specified eight-instruction ISA subset across a fully gate-level ALU, a Big-Endian memory system, and a standard five-element datapath (PC, Instruction Memory, Register File, ALU, Data Memory) with a control unit and three multiplexers. Simulation results confirm that arithmetic, logical, memory, and branch instructions all produce the expected register and memory states within the prescribed testbench environment.

While the current single-cycle architecture successfully meets all design requirements, the project lays a strong foundation for several future enhancements. To significantly improve performance and instruction throughput, the datapath could be upgraded to a classic 5-stage pipelined architecture. Additionally, the supported instruction set could be expanded to include more complex control flow mechanisms, such as `bne` (branch not equal) and jump instructions (`jal`, `jalr`), alongside a broader array of standard RISC-V operations, evolving the design into a more robust and comprehensive processor.